

Sprint Plan – Sprint 2 Plan – VeriFi – Completion: Jul 14, 2026

Goal:

Develop the core functionality of each system component independently so they can be integrated in the following sprint. Each component (database, frontend, API/Server) should be implemented and tested before integration.

Story Point Scale (Fibonacci)

Points

- | | |
|----|---|
| 1 | Trivial — under an hour, no uncertainty |
| 2 | Small — straightforward, well-understood |
| 3 | Medium — some complexity or minor unknowns |
| 5 | Large — meaningful effort or real uncertainty |
| 8 | Very large — significant complexity or dependency risk |
| 13 | Extra large — major unknowns; should be broken down if possible |

Spikes (6):

- **Spike 3 (Vector Database) (3):** Figuring out the mathematics to find K-Best chunks of data. Figuring out how to further optimize the algorithm to support $O(N)$ time.
- **Spike 4 (API/Backend) (3):** Research FastAPI framework and how to connect to frontend/database.
- **Spike 5 (Prompting + QA) (3):** Research Retrieval-Augmented Generation (RAG), prompt engineering techniques, and methods for evaluating AI-generated responses. Design an initial prompt template and establish quality guidelines for future testing.

Task Listing (Priority Ordered):

User story 1.1

[As a user of the new Verifi project I want to go through a demo or mvp version of the project]

- **Task 1:** Create a basic functional webpage that users can move through(3)
- **Task 2:** Have surface level integration and responses to user queries (3)

Total for User Story 1.1: 6 Story points

User story 1.3

[As a system administrator, I want to be able to upload raw financial policy text files into the system, so that the custom semantic vector database is populated with the necessary knowledge to process user queries.]

- **Task 1:** Create a functional small parser that can take in .txt/.pdf files in order to test out parsing capabilities and assess how we want to pass in the chunked information to the backend (3)
- **Task 2:** Create test functions to ensure information is being received on both ends (1)
- **Task 3:** Build a small database where the parsed information is stored, initially without cosine-similarity (1)

Total for User Story 1.3: 5 Story Points

User story 1.2

[As a Fidelity user, I want the chatbox to provide a clickable link to the source document alongside my answer, so that I can verify the information myself.]

- **Task 1:** Figure out a way to extract the links to the documents and structure them within the .jsonl file for referencing within the backend later on in the pipeline (3)
- **Task 2:** Research how retrieved document chunks, metadata, and citations should be formatted before being passed to the LLM for response generation (3)

Total for User Story 1.2: 6 Story Points

User story 1.4

[As a front-end developer, I want a running /chat endpoint with a defined request/response contract, so that I can begin building the UI against mocked data without waiting on the full pipeline.]

- **Task 1:** Set up a Fast API project w/ venv and dependency management, and verify team machine compatibility (3)
- **Task 2:** Implement GET /health returning {"status": "ok"} with HTTP 200 (1)
- **Task 3:** Implement POST /chat accepting {"query": string} and returning a hardcoded mock response: {"answer": string, "sources": [{doc, page, snippet, score}]} (1)

- **Task 4:** Configure CORS middleware to allow the React frontend on localhost:3000 to communicate with the API (2)
- **Task 5:** Write and share the API contract documentation (request schema, response schema, error format, endpoint descriptions) with all teammates (1)

Total for User Story 1.4: 10 story points

User Story 2.1

[As a programmer maintaining the project, I want a clean file structure and commentated function so that it is easier to update and improve the functionality / pipeline.]

- **Task 1:** Meet with teammates over Discord call to discuss potential file structure and any setbacks in order to prevent merging issues within Git before any further development.

User Story 2.2

[As a user of the program, I want to see a website design that does not overcomplicate things for me so that I can ask my questions and receive my answer without any hurdles]

- **Task 1:** Again, look at existing sites with chatbots and improve upon the current design skeleton.
- **Task 2:** Show a demo version of the user interface to the public to gain feedback & criticism.

User story 2.3

[As a Fidelity user, I want the AI to answer my financial policy questions using only the information retrieved from official company documents so that I receive accurate, verifiable and trustworthy responses.]

- **Task 1:** Design the initial system prompt that instructs the LLM to answer only from the retrieved document context (3)
- **Task 2:** Create a prompt template that combines the user's question with the retrieved document chunks before sending them to the LLM (2)
- **Task 3:** Test the prompt using sample financial policy documents and refine it to reduce incorrect or unsupported responses (3)

Total for User Story 2.3: 8 story points

User story 2.4

[As a developer, I want a testing plan for VeriFi so that the system can be evaluated consistently as new features are added.]

- **Task 1:** Create a set of test questions covering common financial policy topics and edge cases (2)
- **Task 2:** Develop QA test cases with expected outputs for each test case and citation requirements (3)
- **Task 3:** Verify prompt outputs and API responses follow the expected response format and document any issues found (2)
-

Total for User Story 2.4: 7 story points

Team roles:

Christopher:	Vector Database {Product Owner, Developer}
Sai:	Frontend {Developer}
Divya:	Prompting + QA {Developer}
Srushti:	Data Ingestion {Developer}
Ethan:	Backend + API {Developer, Scrum Master}

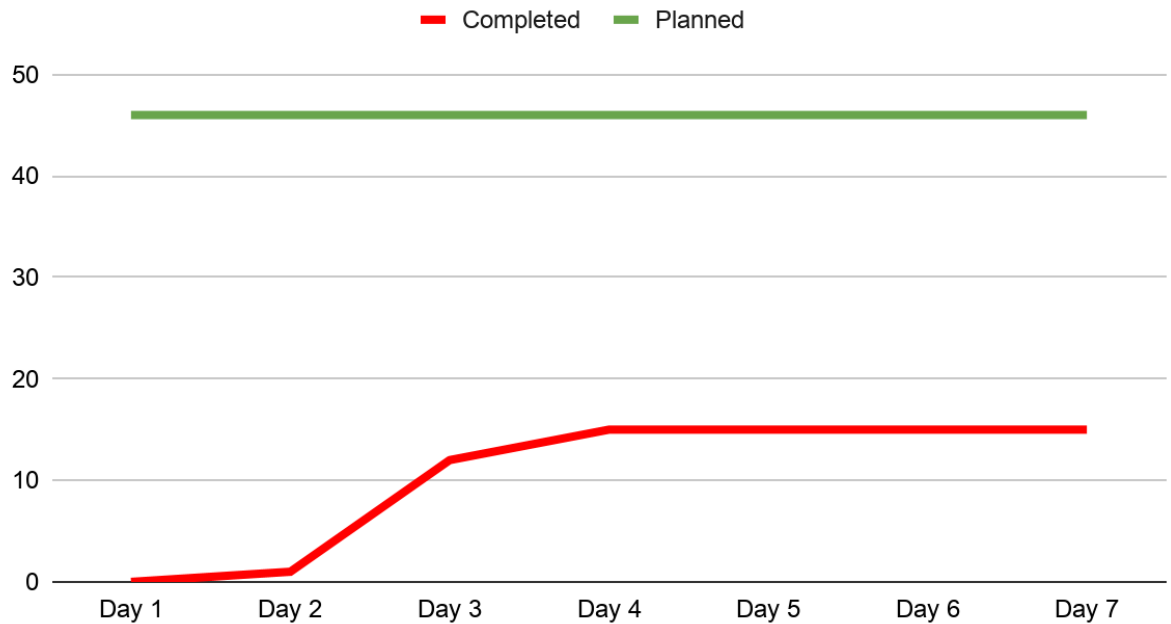
Initial task assignment:

Ethan:	User Story 1.4, Task 1
Christopher:	User Story 1.3, Task 3
Sai:	User Story 1.1 Task 1 & 3
Divya:	User Story 2.3 Task 1 - 3
	User Story 2.4 Task 1 - 3
Srushti:	

Initial burnup chart: A graph giving the initial burnup chart for this sprint and is labeled as such with sprint number and project name and is located in the lab.

Sprint 2 Burnup Chart: Total planned work = 46 story points

Points scored



Initial scrum board: Also known as a task board, the scrum board is a physical board labeled as such with sprint number and project name and located in the lab. This board has four columns, titled user stories, tasks not started, tasks in progress, and tasks completed. Index cards or post-it notes representing the user stories and the tasks for this sprint should be placed in the user stories, tasks not started, and tasks in progress columns. Tasks associated with a user story should be placed in the same row as the user story.

User Stories	Tasks Not Started	Tasks In Progress	Tasks Completed
Spikes (12)		Research Vector Database	- Research REST + FastAPI - Research Prompt Engineering - Research RAG - Design Initial Prompt Template
Story 1.1 (5)			- User Interface Site Design - Submission button/textb

			OX
Story 1.2 (6)		• Formatting parsed information into a .jsonl file / structure	
Story 1.3 (5)	• Write tests for stored embeddings • Create database without cosine-similarity		
Story 1.4 (8)		• Write API Contract	• Set-up dev env for FastAPI • Implement GET/HEALTH • Implement POST/CHAT
Story 2.1 (1)			• Met with teammates to discuss file structure
Story 2.2		• Created polls on Instagram stories to gain feedback from peers and friends – awaiting on more results	
Story 2.3 (8)			• Design system prompt • Prompt Template • Test prompt with sample financial policy documents
Story 2.4 (7)			• Create QA

			test questions ● Define expected outputs ● API and Prompt validation planning
--	--	--	--

Scrum times:

Wednesday (7/1)	Friday (7/3) TA	Saturday (7/4) Team Meeting
3:00 PM	10:00 AM	10:00 AM

Last modified: 2026 07 01